

Research Article

Open Access



A Protected Progressive Adaptive TTL Mechanism for Multi-Robot Task Allocation Under Limited Information Sharing

Zi'an Fang¹, Yuchen Ye¹, Yingxue Du², Wentao Zhang³, Dandan Li¹

¹School of Robotics and Intelligent Manufacturing, Huzhou Normal University, Huzhou 313000, Zhejiang, China.

²School of Automation and Electrical Engineering, Linyi University, Linyi 276000, China

³School of Artificial Intelligence and Robotics, Hunan University, Changsha 410082, China

Correspondence to: Dandan Li

Received: First Decision: Revised: Accepted: Published:

Academic Editor: Copy Editor: Production Editor:

Abstract

Multi-robot systems are now being applied in an increasing number of fields. However, limited communication infrastructure in practical scenarios may require decentralized operation with local information sharing. Therefore, local communication mechanisms play a critical role in information sharing among robots. Yet as the operational area scales up, information sharing grows increasingly challenging. In particular, fixed time-to-live (TTL) strategies cannot dynamically adjust the propagation range to the requirements of individual task, which may in turn reduce the efficiency of task allocation. To address this issue, this paper proposes a protected progressive adaptive time-to-live mechanism, namely PP-AT, for communication-efficient multi-robot task allocation under limited information sharing. When a task is released, the robot that first discovers the task becomes the source node. The source node calculates the TTL upper bound of the task based on the collected task state and local robot state. In the expansion phase, task information is propagated starting from a TTL value of 1, and the propagation range is gradually widened only when the number of feasible candidate robots is insufficient. A protection mechanism leverages state refreshment and assignment awareness to maintain candidate information and guide subsequent propagation. This expansion process terminates once the candidate set meets the feasibility requirement or the TTL attains its predefined upper limit. The proposed method is evaluated within NetFlood + minimum pick-up distance minimization (MPDM) framework and benchmarked against the fixed-TTL baseline. Simulation results reveal that the proposed method achieves substantial communication overhead reduction under the tested scenarios, while maintaining the relative variation in completed task count within 1%.

Keywords: Multi-robot task allocation, limited information sharing, adaptive TTL, communication overhead



© The Author(s) 2024. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License (<https://creativecommons.org/licenses/by/4.0/>), which permits unrestricted use, sharing, adaptation, distribution and reproduction in any medium or format, for any purpose, even commercially, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license, and indicate if changes were made.



1. INTRODUCTION

In recent years, multi-robot systems have been widely applied in logistics, search and rescue, intelligent manufacturing, environmental monitoring, and other scenarios. Compared with manual planning or single-robot systems, multi-robot systems can offer advantages in efficiency and robustness. Related studies cover cooperative exploration under communication constraints, centralized and distributed task allocation, and AI-supported perception, decision-making, and interaction control^[1–5]. As application requirements expand, robots need to adapt their behavior to changes in the environment, task requirements, and the states of their collaborators. This makes coordination among information acquisition, resource allocation, and task execution an important issue in robot cooperation.

For resource-constrained task allocation, Chen and Sun^[7] proposed a coalition-based approach that combines the resources of multiple robots to perform tasks that individual robots cannot easily complete alone. Sequential and holistic coalition strategies are used for online and offline allocation, respectively. Zhang et al.^[8] proposed an extended performance impact algorithm for critical tasks under limited task capacity. By adjusting task lists, the algorithm seeks to maximize the number of critical tasks assigned under capacity constraints. In personalized manufacturing, Huang et al.^[6] proposed a reconfigurable multirobot cooperation workcell. Its modular design separates repetitive and personalized operations and optimizes the relative poses between modules to accommodate the manufacturing requirements of different products. For cooperative perception, Huang et al.^[9] proposed RecSLAM, a system based on a robot–edge–cloud architecture. It combines hierarchical map fusion with adaptive computation offloading while considering load balancing among edge servers to reduce processing latency in multirobot simultaneous localization and mapping. For autonomous task decision-making in dynamic environments, Li et al.^[10] combined large language models with temporal logic to develop an environment-driven task inference and allocation framework. By monitoring changes in environmental resources, generating and verifying tasks, and searching for feasible allocation plans, the framework enables robots to adapt their tasks to environmental needs. In addition, Zhang et al.^[11] proposed an effective iterated greedy algorithm for task allocation and scheduling among agricultural robots in smart farms, with the objective of minimizing the makespan. The algorithm uses the HPF2 strategy to construct an initial solution and adjusts task allocation through destruction and greedy reconstruction. Local search with swap and insertion neighborhoods further improves task execution sequences and task assignments among robots. These studies offer different approaches to improving cooperation efficiency in robotic systems through resource matching, cooperation organization, computation offloading, and task scheduling. Multi-robot task allocation (MRTA) is usually regarded as a core problem in multi-robot systems. Its objective is to optimally allocate tasks or goals to the most appropriate robots according to metrics including task urgency and required execution time.

Existing studies have examined communication constraints and localized information sharing in MRTA. Wang et al.^[12] studied the effects of communication topology, communication range, and perception range on distributed task allocation. Nayak et al.^[13] used common TTL settings to examine the effects of limited information sharing on different allocation strategies. However, since task requirements and the availability of local candidates vary across scenarios, a uniform propagation range may not be suitable for all tasks. This motivates task-specific control of message propagation according to task states and local robot information.

In localized information sharing, TTL is a key factor because it determines the range of information sharing. Specifically, TTL is a mechanism that controls the number of hops for task-message propagation. However, a fixed TTL applies the same propagation range to all tasks, making it difficult to adapt to different task states, local robot distributions, and robot load conditions. When TTL is small, the communication overhead is low, but task visibility is limited, and the system may fail to find enough candidate robots. When TTL is large, task visibility is improved, but a large number of redundant messages may be generated, which reduces communication efficiency.

To tackle this problem, this paper presents a protected progressive adaptive TTL scheme, which dynamically controls TTL according to task states and local robot states. The proposed method calculates a task-specific TTL upper bound based on four local state factors, and the communication range is progressively expanded only when the current range cannot provide sufficient feasible candidate robots. In this way, it eliminates the need to initialize every task with a large TTL while permitting expansion in cases where one-hop information is insufficient. During the progressive TTL expansion process, a protection mechanism is further introduced. This mechanism uses state refresh and assignment awareness to update candidate information and guide subsequent propagation decisions. Its purpose is to reduce the risk of making allocation decisions based on outdated reports and limit unnecessary communication.

The proposed mechanism is integrated into a NetFlood-based localized information sharing framework and evaluated with an MPDM-based task allocation method. Fixed TTL values from 1 to 5 are used as baseline methods. The evaluation metrics include the number of transmitted messages (MSG), the number of completed tasks (NTC), the message cost per completed task (MSG/NTC), total travel delay (TTD), and make-span time (MST). Experimental results show lower communication overhead relative to fixed TTL with comparable mean task completion in the tested settings.

The main contributions of this paper are summarized as follows:

- A state-aware TTL upper bound estimation mechanism is proposed to adapt the communication range to task urgency, resource demand, candidate availability, and local load distribution.
- A protected progressive expansion mechanism is designed to reduce unnecessary message propagation while considering task allocation feasibility.
- The proposed PP-AT mechanism is integrated with task allocation algorithms under a limited information sharing framework and evaluated against fixed TTL. Experimental results demonstrate that PP-AT can effectively reduce communication messages while maintaining comparable task completion performance, providing a practical trade-off between communication cost and allocation reliability.

The remainder of this paper is organized as follows. Section 2 presents the related work and problem formulation. Section 3 introduces the proposed PP-AT method. Section 4 describes the experimental settings and result analysis. Section 5 discusses the advantages and limitations of the proposed method, as well as the special case where the fixed TTL equals 1. Finally, Section 6 draws concluding remarks and outlines future research directions.

2. RELATED WORK AND PROBLEM FORMULATION

This section reviews existing studies on multi-robot task allocation and limited information sharing. Building on this literature, the problem investigated in this paper is formulated within a localized communication framework. The overall objective of this section is to situate the research in its broader context and define the communication-control problem that the proposed PP-AT mechanism aims to address.

2.1. Multi-Robot Task Allocation

Existing MRTA methods aim to optimize various performance objectives, including task completion, travel distance, makespan or service time, energy consumption, and workload balance. Representative approaches include centralized optimization, distributed auction-based methods, heuristic and metaheuristic methods, and learning-based methods. These approaches differ not only in their optimization objectives and solution methods but also in the task structures and constraints they consider.

For multi-objective task allocation, Yu et al.^[14] proposed a multimodal multiobjective optimization method combining deep reinforcement learning and large neighborhood search. Their method treats task allocation and path planning as a bi-level problem to provide multiple equivalent optimized solutions and improve computational efficiency. Martin et al.^[15] used iterative clustering to divide the global allocation problem into smaller subproblems and optimize robot-task assignments within each cluster, addressing tasks that require collaboration among heterogeneous robots. Dai et al.^[16] trained a decentralized task-selection policy through reinforcement learning. By considering robot capabilities and scheduling dependencies, their method reduces idle waiting during cooperation and shortens the overall completion time. In addition, Zhang et al.^[17] addressed time-window and task-precedence constraints through task batching and a batch solver, reducing the number of robots required and the energy consumed during task execution.

Although these studies improve MRTA through optimization, collaborative organization, and scheduling, their performance should be interpreted in light of their respective information-access assumptions. The task and robot states accessible to an allocator depend on the information-sharing mechanism. Accordingly, the impact of communication range and local information on task allocation merits thorough investigation.

2.2 Limited Information Sharing and TTL-Based Communication

The communication mechanism determines what information robots can obtain during task allocation. It is important to distinguish decentralized decision-making from localized information sharing. Dai et al.^[16] proposed a reinforcement learning method for heterogeneous multi-robot task allocation and scheduling, in which each robot

independently selects its subsequent tasks. However, its execution still relies on global communication to collect broadcast information about task selections and robot working states. Therefore, discussions of decentralized task allocation methods should also clarify their information-sharing scope and communication assumptions.

For distributed task allocation, Wang et al.^[12] proposed an effective and efficient performance impact (EEPI) algorithm. With an improved cost function and task release procedure, the method yields more task allocations satisfying both deadlines and robot fuel constraints. Comparative experiments are conducted across varying communication topologies, communication ranges, and perception ranges. The results indicate that inadequate communication range may hinder direct or indirect exchange of allocation information among some robots, potentially resulting in duplicate task assignments. These findings indicate that information-access and exchange conditions affect both task visibility and allocation consistency among robots. The main improvements concern task allocation and convergence, rather than dynamically controlling message propagation hops for individual tasks.

Nayak et al.^[13] further investigated the effects of limited information sharing on multi-robot task allocation. In their framework, task information, robot states, and allocation results are disseminated through NetFlood, with a common TTL setting used to limit the propagation range. The study compared MPDM, RBTS, and MARL with their localized versions and varied TTL to examine the relationship between communication message counts and task allocation performance. Using common TTL settings to compare different propagation ranges, the results showed that a larger TTL increases message transmission but does not necessarily provide corresponding gains in task completion in the tested scenarios.

Building on these studies, this paper focuses on controlling task-message propagation according to allocation needs. Instead of applying a common TTL setting to all tasks, PP-AT estimates a propagation upper bound from the task state and the source robot's local information. Propagation starts from one hop and expands progressively, with candidate states and allocation results informing subsequent propagation decisions. Rather than replacing the existing task allocator, this mechanism operates within a localized information-sharing framework by adjusting message propagation, thereby reducing unnecessary communication while evaluating its effects on task completion performance.

2.3 Problem Formulation

The problem considered in this paper is based on a decentralized multi-robot task allocation system in a two-dimensional workspace. The robot set is denoted as $\mathcal{R} = \{r_1, r_2, \dots, r_m\}$, and the task set is denoted as $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_n\}$. Tasks are released dynamically over the course of a mission, with allocation decisions drawn from locally available information. For each robot r_i , its state at time t is defined as

$$r_i(t) = (p_i(t), q_i(t), s_i(t), l_i(t))$$

where $p_i(t)$ denotes the position of robot r_i , $q_i(t)$ denotes its remaining resource, $s_i(t) \in \{\text{idle}, \text{busy}\}$ denotes its working state, and $l_i(t)$ denotes its current load. For each task τ_j , the state is defined as

$$\tau_j = (p_j, a_j, \rho_j, q_j, E_j)$$

where p_j denotes the task position, a_j denotes the release time, ρ_j denotes the task priority, q_j denotes the required resource, and E_j denotes the execution time. A task is deemed complete once the assigned robot arrives at the task location, finishes its execution, and consumes the required resource.

The communication relationship among robots is modeled as a dynamic graph:

$$G(t) = (\mathcal{R}, \mathcal{E}(t))$$

An edge exists between two robots if their Euclidean distance is not larger than the communication radius d_c :

$$(r_i, r_k) \in \mathcal{E}(t) \quad \text{if } \|p_i(t) - p_k(t)\| \leq d_c$$

After a task τ_j is released at a_j , the robot that first discovers it at $t_j \geq a_j$ becomes the source node, denoted as s_j . The task message is then propagated through the communication graph by NetFlood. The propagation range is controlled by the TTL value h_j , where $h_j \in \{1, 2, 3, 4, 5\}$. At decision time t , an idealized feasible candidate set

based on the current communication graph and actual robot states is

$$C_j^{\text{ideal}}(h_j, t) = \left\{ r_i \in \mathcal{R} \left| \begin{array}{l} H_{s_j i}(t) \leq h_j \\ s_i(t) = \text{idle} \\ q_i(t) \geq q_j \end{array} \right. \right\}$$

where $H_{s_j i}(t)$ denotes the shortest hop distance in $G(t)$. This idealized set describes the basic communication, state, and resource requirements. the source node does not directly observe all actual robot states. During progressive propagation, movement and report delays can make the received information differ from this instantaneous set. Accordingly, PP-AT leverages the reported-state sets defined in Section 3.3 to inform its decisions, incorporating an estimated completion-time check into the allocation process. The selected robot checks its own availability and resource before accepting an assignment. The key notations and their descriptions used hereafter are listed in Table 1.

Table 1. Notation Descriptions

Notation	Description
$\mathcal{R} = \{r_1, r_2, \dots, r_m\}$	Set of robots
$\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_n\}$	Set of tasks
r_i	The i th robot
τ_j	The j th task
$p_i(t)$	Position of robot r_i at time t
$q_i(t)$	Remaining resource of robot r_i
$s_i(t)$	Working state of robot r_i
$l_i(t)$	Current load of robot r_i
p_j	Position of task τ_j
a_j	Release time of task τ_j
ρ_j	Priority of task τ_j
q_j	Required resource of task τ_j
E_j	Execution time of task τ_j
t	Current decision time
H_j	TTL upper bound of task τ_j
h_j	Current propagation TTL of task τ_j
$G(t)$	Dynamic communication graph among robots
$\mathcal{E}(t)$	Set of communication edges at time t
d_c	Communication radius of each robot
s_j	Source robot that first discovers task τ_j
t_j	Discovery time of task τ_j
$H_{s_j i}(t)$	Shortest communication hop distance from s_j to r_i at time t
$C_j^{\text{ideal}}(h_j, t)$	Idealized feasible candidate set at time t
N_{TC}	Number of completed tasks
M_{SG}	Number of transmitted messages
$M_{\text{SG}}/N_{\text{TC}}$	Messages per completed task

To ensure a fair comparison with the localized information sharing framework presented in the reference study, the main evaluation metrics are adopted from the the baseline study^[13], including NTC, MSG, TTD, and MST. Their aggregation rules in this implementation are specified in Section 4.1. In addition, MSG/NTC is further introduced as a derived metric to evaluate the communication cost required for each completed task. It should be noted that the proposed PP-AT algorithm is not a new task allocator intended to replace allocators such as MPDM. Instead, PP-AT operates as a communication-control mechanism that regulates the propagation range of task messages prior to the execution of task allocation decisions. In summary, the problem studied in this paper is to determine a task-specific TTL for each dynamically released task under localized information sharing, so that communication overhead can be reduced while maintaining the number of completed tasks.

3. PROPOSED METHOD

This section introduces the protected progressive adaptive TTL method, namely PP-AT. PP-AT is not intended to serve as a new task allocator. Instead, the TTL value, which governs the propagation range of task messages, is determined before the task allocator takes over, so as to reduce communication overhead while preserving task

completion throughput.

3.1 State-Aware TTL Upper Bound Estimation

For each dynamically released task, PP-AT first estimates the required TTL upper bound according to the task state observed by the source robot and the robot states within the one-hop communication range of the source robot. This TTL upper bound determines the maximum propagation range allowed during message expansion. The purpose of this design is to avoid propagating every task message throughout the entire local network, while permitting expansion of the propagation range when the number of candidate robots covered by the current TTL is inadequate. For task τ_j , the one-hop neighborhood of the source robot s_j is defined as

$$\mathcal{N}_1(s_j) = \{r_i \in \mathcal{R} \mid \|p_i(t_j) - p_{s_j}(t_j)\| \leq d_c\}$$

Based on the one-hop neighborhood information obtained from $\mathcal{N}_1(s_j)$, four state factors, namely P_j , R_j , C_j , and L_j , are selected to calculate the TTL upper bound. These four factors describe the task urgency, resource demand, candidate sufficiency, and local load imbalance, respectively. A higher P_j or R_j indicates that the task may require a larger propagation range, while a higher C_j means that sufficient feasible robots are already available in the one-hop neighborhood. In addition, a higher L_j indicates a more imbalanced local workload, which may require a larger search range to improve allocation reliability. The detailed definitions of these factors are listed in Table 2.

Table 2. State Factors Used in TTL Upper Bound Estimation

Symbol	Description	Range
P_j	Normalized priority of task τ_j	$[0, 1]$
R_j	Normalized resource demand of task τ_j	$[0, 1]$
C_j	Candidate robot sufficiency in $\mathcal{N}_1(s_j)$	$[0, 1]$
L_j	Local load imbalance in $\mathcal{N}_1(s_j)$	$[0, 1]$
$N_{\text{candidate}}$	Number of feasible candidate robots in $\mathcal{N}_1(s_j)$	$\mathbb{Z}_{\geq 0}$
q_j	Required resource of task τ_j	$[r_{\min}, r_{\max}]$
$r_{\min}$	Minimum task resource demand	positive scalar
$r_{\max}$	Maximum task resource demand	positive scalar
N_{ref}	Reference number of sufficient candidate robots	positive scalar
ϵ	Small constant used to avoid division by zero	positive scalar
θ_0	Basic propagation demand coefficient	scalar
θ_1	Weight of task priority	scalar
θ_2	Weight of task resource demand	scalar
θ_3	Weight of candidate sufficiency	scalar
θ_4	Weight of local load imbalance	scalar
load	Load vector of robots in $\mathcal{N}_1(s_j)$	vector
std(load)	Standard deviation of local robot loads	non-negative scalar
mean(load)	Mean value of local robot loads	non-negative scalar

The four state factors are normalized to $[0, 1]$ as follows.

$$P_j = \text{clip}_{[0,1]}(\rho_j)$$

$$R_j = \text{clip}_{[0,1]} \left(\frac{q_j - r_{\min}}{r_{\max} - r_{\min}} \right)$$

$$C_j = \text{clip}_{[0,1]} \left(\frac{N_{\text{candidate}}}{N_{\text{ref}}} \right)$$

$$L_j = \text{clip}_{[0,1]} \left(\frac{\text{std}(\text{load})}{\text{mean}(\text{load}) + \epsilon} \right)$$

The reference count N_{ref} normalizes candidate sufficiency. It does not set the expansion stopping threshold. A small constant $\epsilon > 0$ prevents division by zero. Clipping is defined by

$$\text{clip}_{[a,b]}(x) = \min\{b, \max\{a, x\}\} \quad (1)$$

$$a \leq b$$

These factors determine the task-specific TTL upper bound through a weighted score:

$$H_j = \text{clip}_{[1,5]}(\text{round}(\theta_0 + \theta_1 P_j + \theta_2 R_j - \theta_3 C_j + \theta_4 L_j))$$

The coefficient θ_0 sets the base score, and θ_1 – θ_4 weight priority, resource demand, candidate sufficiency, and load imbalance, respectively. Parameter values are reported in the experimental settings.

3.2 Protected Progressive Expansion

PP-AT starts propagation at $h_j = 1$ and uses H_j as the expansion limit. After each round, the source node checks candidate feasibility from received robot-state reports and decides whether to allocate, wait, or expand. Let $\mathcal{V}_j(h_j, t)$ denote the set of robots that are visible under TTL h_j with valid state reports at the source. For robot r_i , these reports give its position $\hat{p}_i$, remaining resource $\hat{q}_i$, and busy-until time $\hat{B}_i$. Its estimated task completion time is

$$\hat{T}_{ij}(t) = \max\{\hat{B}_i, t\} + \frac{\|\hat{p}_i - p_j\|}{v} + E_j \quad (2)$$

Here, v is the robot speed and E_j is the execution time. Because reports may become outdated, this estimate does not guarantee completion. Robots with sufficient resources and estimated completion by the simulation end time T_{end} form

$$\mathcal{F}_j(h_j, t) = \left\{ r_i \in \mathcal{V}_j(h_j, t) \left| \begin{array}{l} \hat{q}_i \geq q_j \\ \hat{T}_{ij}(t) \leq T_{\text{end}} \end{array} \right. \right\} \quad (3)$$

Among these robots, those reported as idle form the immediately feasible candidate set:

$$C_j^{\text{imm}}(h_j, t) = \{r_i \in \mathcal{F}_j(h_j, t) \mid \hat{B}_i \leq t\} \quad (4)$$

Propagation stops and task allocation begins once

$$|C_j^{\text{imm}}(h_j, t)| \geq 1 \quad (5)$$

Otherwise, PP-AT considers waiting before expanding. Task priority determines the waiting threshold:

$$W_j = W_{\text{max}} - (W_{\text{max}} - W_{\text{min}})P_j \quad (6)$$

The bounds W_{min} and W_{max} specify the allowed threshold range. The shortest reported wait within $\mathcal{F}_j(h_j, t)$ is

$$w_j(t) = \begin{cases} \min_{r_i \in \mathcal{F}_j(h_j, t)} \max\{\hat{B}_i - t, 0\} & \mathcal{F}_j(h_j, t) \neq \emptyset \\ +\infty & \mathcal{F}_j(h_j, t) = \emptyset \end{cases} \quad (7)$$

For $w_j(t) \leq W_j$, PP-AT retains the current TTL and rechecks at $t + \max\{\Delta_c, w_j(t)\}$, with Δ_c denoting the collection interval. Setting $W_{\text{min}} = 10$ s and $W_{\text{max}} = 60$ s gives $W_j = 60 - 50P_j$ seconds.

When $C_j^{\text{imm}}(h_j, t) = \emptyset$, $w_j(t) > W_j$, and $h_j < H_j$, propagation expands by one hop:

$$h_j \leftarrow h_j + 1 \quad (8)$$

A further $2\Delta_c$ interval allows message propagation and state collection before reassessment. At $h_j = H_j$, waiting remains possible when $w_j(t) \leq W_j$. Otherwise, the controller terminates without implying successful allocation or completion. These rules limit unnecessary propagation while allowing a wider search when local candidates cannot serve the task promptly.

The protection mechanism also maintains the state reports used in these decisions. Let $\widehat{t}_i$ be the timestamp of a stored report. A visible remote robot with reported resource $\widehat{q}_i \geq q_j$ refreshes its state when

$$t - \widehat{t}_i \geq A_{\max} \quad (9)$$

The refresh uses NetFlood with the current TTL and is included in StateMSG. The report is updated only if it reaches the source; otherwise, the robot is removed from the current visible and reported sets. The source updates its own state locally. In addition, sources reached by an existing allocation-result dissemination update the reported busy-until time and load of the assigned robot for other pending tasks that already contain its report. Their decisions are then scheduled for reassessment after Δ_c . This reuses the counted allocation dissemination rather than assuming a system-wide state update.

To eliminate redundant retransmissions during the expansion phase, each task message maintains a reached-node set and a forwarding history. Only nodes on the current propagation frontier that have not forwarded this task message send it to their current neighbors. Each node forwards a given task message at most once during flooding, and every outgoing transmission counts toward communication overhead, including copies delivered to nodes already aware of the task. State reports are cached between expansion rounds, with stale reports refreshed as described above. Consequently, the reached set depends on the propagation history and need not equal the instantaneous hop neighborhood in $G(t)$.

3.3 Integration with MPDM

Following the localized information-sharing framework of the reference study, this paper adopts MPDM + NetFlood and integrates the proposed PP-AT mechanism for comparison with fixed-TTL baselines. PP-AT is used to determine the propagation range of task messages, while the MPDM allocator is used to complete task allocation among candidate robots that receive the task messages.

For each dynamically released task, the source robot first calculates the TTL upper bound of the task, and the protected progressive expansion mechanism is then applied to determine the final propagation range. After that, MPDM performs task allocation based on the candidate robot set formed within the determined propagation range.

It should be stated that PP-AT controls task-message propagation, waiting, and reported-state updates, and does not change the task allocation rules of MPDM. Therefore, when compared with fixed-TTL methods, the differences in the obtained results mainly come from the change in the communication-control mechanism.

Algorithm 1 summarizes the event-driven communication controller. The controller state is initialized on task discovery and retained for subsequent decision and allocation-result events. Each event updates the corresponding state fields, without necessarily altering all outputs. A ready status indicates that the task is handed to MPDM, which does not guarantee successful allocation or completion.

4. EXPERIMENTS AND RESULTS

This section evaluates the effectiveness of the proposed PP-AT method through simulation experiments. The experiments are conducted based on the NetFlood + MPDM framework, and the PP-AT method is compared against fixed-TTL baseline methods with TTL values ranging from 1 to 5. The evaluation focuses on whether PP-AT can reduce communication overhead while preserving task performance.

4.1 Experimental Setup

The experiments are conducted in a two-dimensional decentralized multi-robot task allocation environment. Upon dynamic task release, task messages are propagated via the NetFlood protocol and task allocation is carried out by the MPDM allocator. To ensure a fair comparison, all methods are evaluated under the same task scenarios, robot settings, and random seeds. In the experiments, the TTL upper bound uses the fixed heuristic coefficients reported in Table 3, together with normalized task resource demand and candidate sufficiency. The evaluation metrics include the number of completed tasks (NTC), the number of transmitted messages (MSG), messages per completed task (MSG/NTC), total travel delay (TTD), and make-span time (MST). The main experimental parameters are listed in Table 3.

The environment follows the workspace geometry, task clusters, robot speed, discovery radius, and execution-time

Algorithm 1 Protected Progressive Adaptive TTL (PP-AT)

Input: Task τ_j (position, priority, resource demand, and execution time), source s_j , locally available state reports, current time t , configuration parameters, and stored controller state (if initialized)

Output: Updated controller state: TTL h_j , report cache, propagation history, next decision time, and controller decision or status

On task discovery:

```

1: Compute  $P_j, R_j, C_j, L_j$  and estimate  $H_j$ 
2:  $W_j \leftarrow W_{\max} - (W_{\max} - W_{\min})P_j; h_j \leftarrow 1$ 
3: Initialize the report cache and persistent flood history
4: Propagate at  $h_j$  using incremental NetFlood and collect reports
5: Schedule the first decision at  $t + 2\Delta_c$ 
On a scheduled decision for a pending task:
6: Update the source's own state locally
7: for each visible remote report with  $\hat{q}_i \geq q_j$  and  $t - \hat{t}_i \geq A_{\max}$  do
8:   Refresh via NetFlood at  $h_j$  and count StateMSG
9:   Update the report if received; otherwise invalidate its membership
10: end for
11: Form  $\mathcal{F}_j(h_j, t)$  and  $C_j^{\text{imm}}(h_j, t)$  using reported states
12: if  $C_j^{\text{imm}}(h_j, t) \neq \emptyset$  then
13:   Mark the controller ready and hand the task to MPDM
14: else
15:    $w_j(t) \leftarrow \begin{cases} \min_{r_i \in \mathcal{F}_j(h_j, t)} \max\{\hat{B}_i - t, 0\} & \mathcal{F}_j(h_j, t) \neq \emptyset \\ +\infty & \mathcal{F}_j(h_j, t) = \emptyset \end{cases}$ 
16:   if  $w_j(t) \leq W_j$  then
17:     Retain  $h_j$  and schedule reassessment at  $t + \max\{\Delta_c, w_j(t)\}$ 
18:   else if  $h_j < H_j$  then
19:      $h_j \leftarrow h_j + 1$ 
20:     Extend the persistent flood and collect additional reports
21:     Schedule reassessment at  $t + 2\Delta_c$ 
22:   else
23:     Mark the controller max_reached; return to the task lifecycle
24:   end if
25: end if
On receiving an existing allocation-result dissemination:
26: for each pending task at a reached source that already stores the assigned robot's report do
27:   Update reported busy-until time, load, and timestamp
28:   Reactivate its controller and schedule reassessment at  $t + \Delta_c$ 
29: end for

```

range stated in the reference study^[13]. The 60-robot, 2-tasks/s case is used as the base setting. Parameters not fully specified in that study are explicit implementation assumptions: a 500-s horizon, a 1-s time step and collection interval, exponential task interarrival times, uniform resource demand in $[8, 25]$, uniform priority in $[0, 1]$, and deterministic random-waypoint exploration by idle robots. Initial robot positions are sampled uniformly over the workspace. Task locations are sampled around the four centers with equal probability and clipped to the workspace boundary. Resource demand and execution time are continuous uniform samples. This is a reference-aligned reconstruction, rather than an exact reproduction of the original simulator. Communication is limited by range, topology, and TTL. The model does not simulate a bandwidth cap, packet loss, MAC contention, or delay jitter.

An auxiliary comparison uses a lightweight shared-policy MARL allocator implemented for this study, rather than an exact reproduction of the reference MARL network. Its actor and critic each have hidden layers of 64 and 32 units. A 65-dimensional observation supports up to ten task choices and a no-operation action. Robots propose tasks from masked action probabilities. Conflicts are resolved by proposal probability, with robot index breaking ties. Training uses 210 episodes of 120 s with mixed visibility settings (Global, Fixed TTL = 1–5, and Adaptive TTL). The saved checkpoint is shared by all six MARL test methods. Test actions are deterministic, with no training or exploration. MPDM uses seeds 750010–75020 and MARL uses 77001–77020. Pairing applies within each group, not between the two allocators. Reported table entries are the mean and sample standard deviation over the 20 seeds.

Following the terminology in the reference study, TTD is the accumulated travel time and MST is the sum of travel and execution times, both measured in seconds. Let $\mathcal{A}_{\text{acc}}$ contain the robot–task assignments accepted during the

Table 3. Experimental Parameters

Parameter	Value	Parameter	Value
Workspace size	1000 × 1000 m	TTL range	[1, 5]
Number of robots	60	Compared methods	Fixed TTL = 1–5, PP-AT
Maximum number of tasks	1000	Number of random seeds	20 per allocator group
Simulation time	500 s	MPDM random seeds	75001–75020
Time step	1 s	MARL random seeds	77001–77020
Task arrival rate	2 tasks/s (exponential interarrival times)	Collection interval Δ_c	1 s
Task release mode	Dynamic release	Waiting thresholds ($W_{\min}, W_{\max}$)	(10, 60) s
Task centers	(250, 250), (700, 250), (250, 700), (700, 700)	State refresh threshold $A_{\max}$	5 s
Task cluster standard deviation	150 m	θ_0	1.7
Communication protocol	NetFlood	θ_1	0.5
Task allocator	MPDM; lightweight MARL (auxiliary)	θ_2	0.4
Communication radius	100 m	θ_3	1.2
Task discovery radius	50 m	θ_4	0.3
Robot speed	10 m/s	N_{ref}	2
Initial robot resource	70	ϵ	10^{-6}
Task resource demand	[8, 25]	Message counting mode	Link copies
Task execution time	[10, 30] s		

simulation, and let d_{ij}^{assign} be the travel distance at acceptance. The implemented aggregates are

$$\begin{aligned} \text{TTD} &= \sum_{(i,j) \in \mathcal{A}_{\text{acc}}} \frac{d_{ij}^{\text{assign}}}{v} \\ \text{MST} &= \sum_{(i,j) \in \mathcal{A}_{\text{acc}}} \left(\frac{d_{ij}^{\text{assign}}}{v} + E_j \right) \end{aligned} \quad (10)$$

Both totals are recorded when an assignment is accepted, including planned travel and execution for accepted tasks unfinished at the horizon. This aggregation convention is stated explicitly because the reference description does not resolve that boundary case. MST here follows the reference's additive definition rather than the maximum completion time across robots. NTC counts only tasks completed by the horizon. For each seed, MSG/NTC is computed from that run's totals, and the table reports the mean of these per-run ratios.

Communication overhead includes all three protocol phases:

$$\text{MSG} = \text{TaskMSG} + \text{StateMSG} + \text{AllocationMSG} \quad (11)$$

TaskMSG counts task announcements and progressive forwarding. StateMSG counts robot-state replies, refreshes, and any retry-related state communication performed by the simulator. AllocationMSG counts allocation-result dissemination with twice the task's current TTL. All phases use the same link-copy rule: one outgoing copy to one adjacent robot counts as one message, so a transmission to k neighbors counts as k messages. This is a count of transmitted copies, not unique payloads or over-the-air broadcast events. The fixed-TTL baselines retain a fixed task-message range. PP-AT further employs incremental forwarding and report maintenance, and the messages associated with these two mechanisms are included in the aforementioned communication totals.

4.2 Comparison With Fixed-TTL Baselines

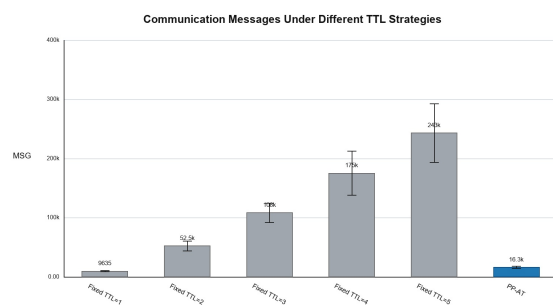
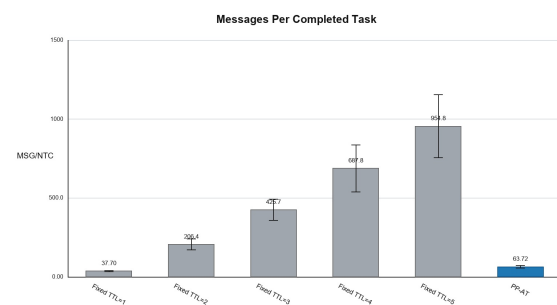
This section first evaluates PP-AT under the MPDM allocator, which is the main experimental setting of this paper. To further check whether the proposed communication-control mechanism depends on a specific allocator, an auxiliary MARL-based comparison is also reported. In both groups, PP-AT is evaluated against five fixed-TTL baselines (TTL = 1–5) under a shared NetFlood-based information dissemination framework.

Table 4 summarizes the main comparison results between PP-AT + MPDM and the fixed-TTL baselines, while Table 5 reports the corresponding auxiliary results under MARL. Within each allocator group, all methods are evaluated with the same task scenarios, robot settings, and paired random seeds.

Table 4. Main Comparison Results Under MPDM

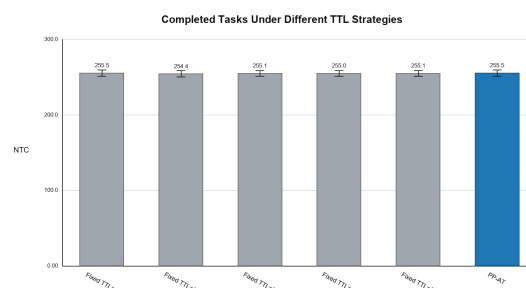
Method	NTC	MSG	MSG/NTC	TTD (s)	MST (s)
Fixed TTL = 1	255.50 $\pm$ 4.27	9634.60 $\pm$ 866.84	37.70 $\pm$ 3.28	1335.10 $\pm$ 69.94	6425.04 $\pm$ 137.68
Fixed TTL = 2	254.45 $\pm$ 4.08	52462.15 $\pm$ 8190.95	206.41 $\pm$ 33.82	1587.00 $\pm$ 47.70	6642.61 $\pm$ 119.98
Fixed TTL = 3	255.10 $\pm$ 3.80	108466.40 $\pm$ 16423.20	425.72 $\pm$ 67.99	1784.20 $\pm$ 98.50	6871.55 $\pm$ 154.18
Fixed TTL = 4	255.00 $\pm$ 3.60	175177.70 $\pm$ 37142.83	687.75 $\pm$ 148.95	1875.50 $\pm$ 103.41	6954.40 $\pm$ 163.34
Fixed TTL = 5	255.05 $\pm$ 3.82	243248.90 $\pm$ 49507.70	954.77 $\pm$ 199.70	1958.91 $\pm$ 126.51	7039.07 $\pm$ 182.17
PP-AT	255.50 $\pm$ 4.29	16269.50 $\pm$ 1956.21	63.72 $\pm$ 7.92	1383.62 $\pm$ 57.81	6476.55 $\pm$ 160.62

As shown in Tables 4 and 5, PP-AT achieves a similar mean NTC to the fixed-TTL baselines under both MPDM and MARL. These mean values alone do not establish statistical equivalence. The corresponding MSG and MSG/NTC comparisons are presented in Figs. 1 and 2 for MPDM, and in Figs. 4 and 5 for MARL. Tables report mean $\pm$ sample standard deviation across 20 paired seeds within each allocator group.

**Figure 1.** MSG comparison under MPDM (mean $\pm$ sample standard deviation).**Figure 2.** MSG/NTC comparison under MPDM (mean $\pm$ sample standard deviation).

The MSG results under MPDM and MARL, shown in Figs. 1 and 4, exhibit the same overall tendency: the number of transmitted messages increases rapidly when the fixed TTL is enlarged from 2 to 5. Compared with these larger fixed-TTL settings, PP-AT substantially reduces redundant message propagation under both allocators. However, fixed TTL = 1 still has the lowest communication overhead because it always uses the minimum propagation range.

To account for differences in completed task counts, MSG/NTC is compared in Figs. 2 and 5. Under both MPDM and MARL, PP-AT achieves a lower mean MSG/NTC than fixed-TTL baselines with TTL values of 2 through 5. Thus, the communication savings remain when measured per completed task. This result does not imply that task completion is unchanged in every run.

**Figure 3.** NTC comparison under MPDM (mean $\pm$ sample standard deviation).

The task completion results in Figs. 3 and 6 show that mean NTC values remain close under both allocators in the tested setting. Together with the communication results, they indicate a favorable average trade-off relative to fixed TTL = 2–5. The consistent trends provide auxiliary evidence for applying PP-AT with the two tested allocators, rather than establishing generalization to all allocation algorithms.

Table 5. Auxiliary Comparison Results Under MARL

Method	NTC	MSG	MSG/NTC	TTD (s)	MST (s)
Fixed TTL = 1	255.75 ± 4.58	7283.00 ± 449.63	28.48 ± 1.78	1548.22 ± 65.38	6707.50 ± 182.96
Fixed TTL = 2	253.45 ± 4.76	45218.70 ± 5193.23	178.35 ± 19.76	2110.20 ± 129.23	7205.69 ± 207.94
Fixed TTL = 3	254.70 ± 5.35	98358.25 ± 16949.81	386.59 ± 68.28	2542.21 ± 157.38	7676.22 ± 217.73
Fixed TTL = 4	253.75 ± 4.96	169334.00 ± 36773.96	666.73 ± 139.85	2867.40 ± 199.43	7974.29 ± 287.87
Fixed TTL = 5	253.80 ± 5.34	269748.65 ± 59998.66	1062.55 ± 233.85	3142.89 ± 214.00	8234.56 ± 307.97
PP-AT	254.90 ± 4.76	13098.45 ± 853.83	51.40 ± 3.51	1588.91 ± 65.10	6719.01 ± 190.04

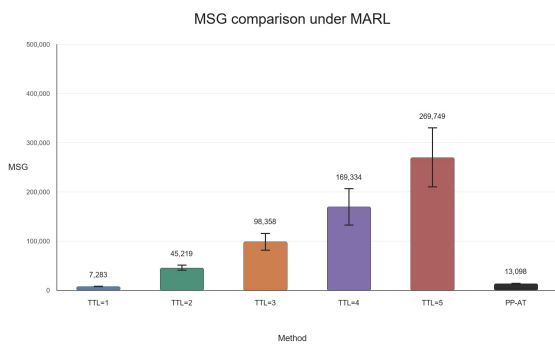


Figure 4. MSG comparison under MARL (mean ± sample standard deviation).

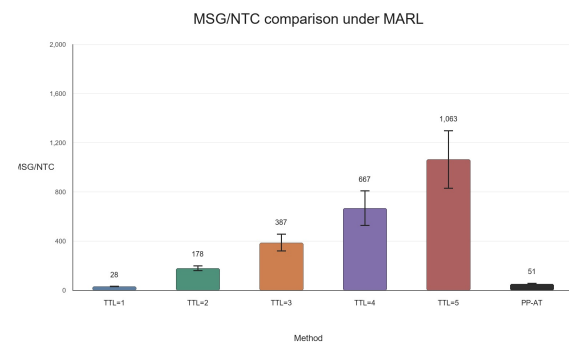


Figure 5. MSG/NTC comparison under MARL (mean ± sample standard deviation).

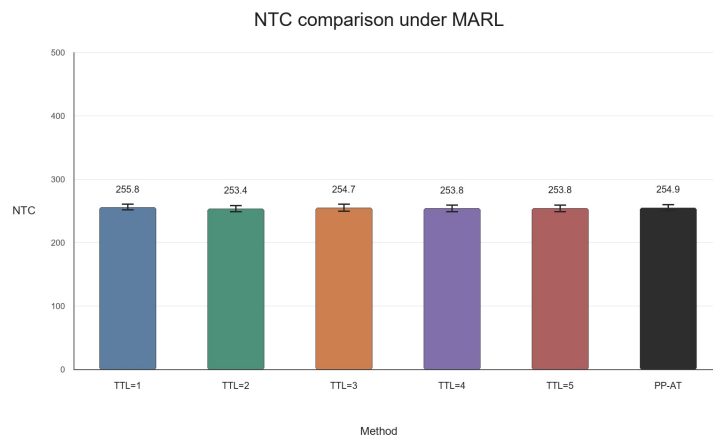


Figure 6. NTC comparison under MARL (mean ± sample standard deviation).

4.3 Ablation and Robustness Analysis

To evaluate the effects of individual components on communication overhead and task completion performance, the ablation experiments adopt the same task scenarios, robot settings, and 20 paired random seeds (76001–76020). All three configurations use the same MPDM allocator and message-counting rules. The service guard, an earlier rule that forces expansion when the estimated completion-time margin is small, is disabled in all configurations. State refresh and assignment awareness are each removed separately from current PP-AT. The results are presented

in Table 6. Reported 95% confidence intervals use paired Student's t intervals calculated from the 20 seed-level differences (19 degrees of freedom), without adjustment for multiple comparisons. They are pointwise intervals, not simultaneous confidence guarantees. Compared with the configuration without state refresh, current PP-AT reduces MSG by 8.75% and MSG/NTC by 8.51%. The mean number of rejected allocation attempts decreases from 110.90 to 25.70, while mean NTC decreases by approximately 0.35%. The paired MSG difference (PP-AT minus the ablated configuration) has a 95% confidence interval of $[-2332.78, -834.42]$. Compared with the configuration without assignment awareness, mean MSG decreases by 4.79%, and the mean number of rejected allocation attempts decreases from 65.40 to 25.70. However, its paired MSG confidence interval, $[-1719.47, 59.17]$, includes zero, so this reduction is not statistically significant. The paired NTC confidence intervals include zero in both comparisons. These results support lower communication overhead with state refresh in this setting and fewer rejected assignments with both components, but do not establish unchanged task completion performance.

To further examine PP-AT under different environments, it is compared with fixed TTL = 1–5 in four settings: the base environment, 100 robots, a task arrival rate of 15 tasks/s, and an initial robot resource level of 30. Each setting uses the same 20 paired random seeds across methods, with identical task scenarios and robot settings within each comparison. This seed set (79001–79020) differs from that used in the main comparison in Table 4. Across the four environments, PP-AT reduces mean MSG by 53.29%–97.69% relative to fixed TTL = 2–5, while mean MSG/NTC also decreases. These results indicate that communication savings remain after normalization by the number of completed tasks in the tested environments. Furthermore, at an arrival rate of 15 tasks/s, PP-AT reduces MSG by 88.05% relative to Fixed TTL = 3, but mean NTC decreases by approximately 0.83%. The comparison results are presented in Table 7 and Fig. 7. Figure 7 presents relative changes calculated from the method means, rather than confidence intervals. The dashed lines mark the 3% MSG reduction and 1% NTC decrease thresholds. Negative MSG reductions indicate higher communication overhead with PP-AT.

Table 6. Direct Component Ablation Results Under MPDM

Configuration	NTC	MSG	MSG/NTC	Rejections
Current PP-AT	253.90 $\pm$ 4.63	16510.45 $\pm$ 1976.49	64.99 $\pm$ 7.28	25.70 $\pm$ 6.62
No state refresh	254.80 $\pm$ 4.58	18094.05 $\pm$ 1801.81	71.03 $\pm$ 7.21	110.90 $\pm$ 14.84
No assignment awareness	253.35 $\pm$ 4.04	17340.60 $\pm$ 2134.22	68.41 $\pm$ 8.03	65.40 $\pm$ 13.85

Values are mean $\pm$ sample standard deviation over 20 paired seeds (76001–76020), with 60 runs in total. All configurations disable the service guard. Current PP-AT enables both state refresh and assignment awareness. Each ablated configuration disables only the named component. Rejections denotes rejected allocation attempts.

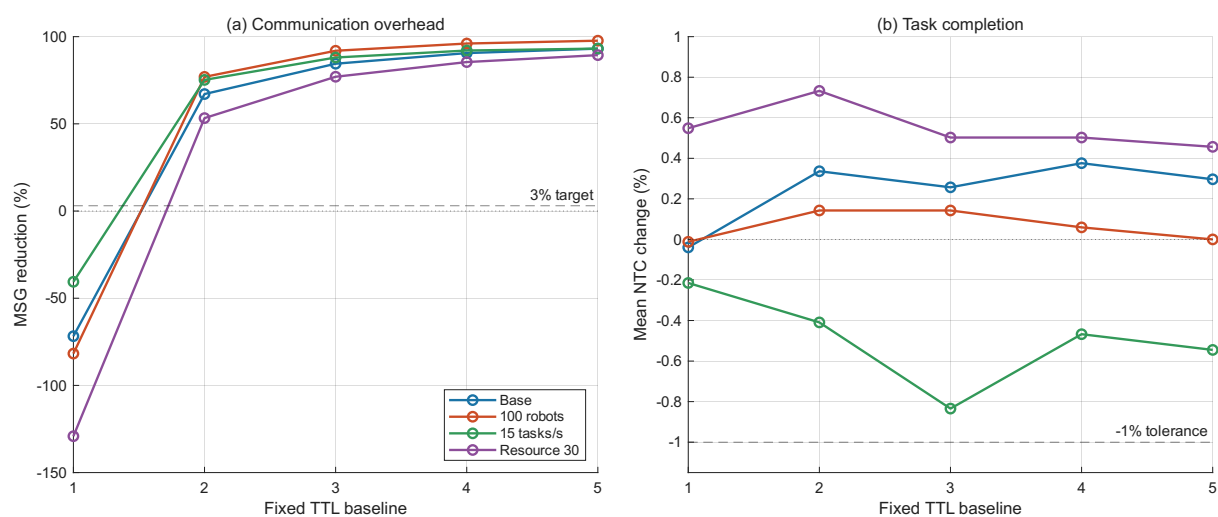


Figure 7. PP-AT versus fixed-TTL baselines across four MPDM environments: (a) MSG reduction; (b) NTC change.

Table 7. PP-AT Relative to Fixed TTL Under Four Environments

Environment	TTL	MSG reduction (%)	NTC change (%)	MSG/NTC reduction (%)	Seed rate
Base	2	67.09	+0.34	67.20	95%
Base	3	84.48	+0.26	84.52	75%
Base	4	90.58	+0.38	90.63	85%
Base	5	93.13	+0.30	93.16	85%
100 robots	2	76.93	+0.14	76.96	90%
100 robots	3	91.92	+0.14	91.93	85%
100 robots	4	96.06	+0.06	96.06	85%
100 robots	5	97.69	+0.00	97.69	75%
15 tasks/s	2	75.21	-0.41	75.09	65%
15 tasks/s	3	88.05	-0.83	87.95	60%
15 tasks/s	4	92.05	-0.47	92.01	70%
15 tasks/s	5	93.17	-0.54	93.13	70%
Resource 30	2	53.29	+0.73	53.54	90%
Resource 30	3	77.01	+0.50	77.09	85%
Resource 30	4	85.43	+0.50	85.48	80%
Resource 30	5	89.41	+0.46	89.43	75%

Each comparison uses 20 paired seeds (79001–79020). Positive reductions indicate lower communication overhead with PP-AT. Negative NTC changes indicate fewer completed tasks. Seed rate is the fraction satisfying $NTC_{PPAT} \geq 0.99 NTC_{Fixed}$. The four cases were selected before this supplement. Eighty archived PP-AT runs are paired with 400 new fixed-TTL runs. These results are based on the four selected environments, and do not include all ten environments used in the earlier AT comparison.

5. DISCUSSION

PP-AT reduces communication overhead relative to fixed TTL = 2–5 in the tested environments. The decrease in MSG/NTC further indicates that these savings remain after normalization by the number of completed tasks. However, PP-AT does not outperform a fixed TTL of 1 in communication overhead. This is because, when the one-hop neighborhood already provides sufficient candidates for allocation, expansion and state maintenance can introduce additional communication without necessarily increasing task completion. Given the substantial communication savings observed relative to the larger fixed-TTL settings, the value of PP-AT lies in adapting propagation to allocation needs rather than achieving the lowest communication overhead in every environment. Moreover, the advantage of a fixed TTL of 1 in these experiments should not be interpreted as a theoretical lower bound on the overhead of all communication mechanisms.

At a task arrival rate of 15 tasks/s, PP-AT reduces MSG by 88.05% relative to fixed TTL = 3, but mean NTC decreases by approximately 0.83%. Although this decrease remains below the predefined 1% tolerance for the mean reduction, it does not imply that task completion performance is entirely unaffected. The 95% confidence interval for the paired NTC difference excludes zero, so this decrease should not be disregarded. Propagation range, changes in candidate states, and competition among tasks may all affect allocation, but the current experiments have not isolated their individual effects. This result demonstrates that a substantial reduction in communication overhead can be achieved, accompanied by only a marginal degradation in task completion performance.

This study has several limitations. The simulation follows the localized information-sharing framework of the reference study^[13], but it does not exactly reproduce all of its experimental details. Furthermore, the robustness comparison against fixed TTL covers only four environments. The direct component ablations cover the base MPDM setting only. Component effects under other environments and allocators remain to be examined. Future work can evaluate the method using a more complete communication model and a broader range of task conditions, and further investigate the sources of communication savings and task completion losses.

6. CONCLUSION

This paper proposes a protected progressive adaptive TTL mechanism, termed PP-AT, for multi-robot task allocation in scenarios with limited information sharing. Rather than replacing existing task allocators, PP-AT functions as a complementary communication-control mechanism. It derives a propagation upper bound from task states and the source robot's local information, progressively extends the propagation radius starting from one hop, and modulates the propagation process via state refresh and assignment awareness. Simulations using NetFlood and MPDM show that PP-AT reduces communication overhead relative to fixed TTL = 2–5 in the tested environments.

These savings remain after normalization by the number of completed tasks. In the robustness comparisons across four environments, any decrease in mean task completion remains below 1%. Auxiliary MARL experiments also provide preliminary evidence that the mechanism can be combined with different allocators. However, PP-AT still incurs higher communication overhead than the fixed TTL of 1, and task completion decreases in some settings. This is because, when the one-hop neighborhood already provides sufficient candidates for allocation, expansion and state maintenance may introduce additional communication without necessarily increasing task completion. The strength of PP-AT therefore lies in its ability to strike a better balance between communication overhead and task performance within the evaluated scenarios, rather than providing consistent improvements across all fixed-TTL configurations. Future work will consider more realistic communication conditions and a broader range of task scenarios to examine the method's applicability.

Financial support and sponsorship

This work was supported by the Open Research Project of the State Key Laboratory of Industrial Control Technology (Grant No. ICT2026B15), and a Project Supported by Scientific Research Fund of Zhejiang Provincial Education Department (Grant No. Y202557528).

REFERENCES

- Amigoni F.; Banfi J.; Basilico N. Multirobot Exploration of Communication-Restricted Environments: A Survey. *IEEE Intell. Syst.* 2017;32(6), 48–57. doi: [10.1109/MIS.2017.4531226](https://doi.org/10.1109/MIS.2017.4531226).
- Geng N.; Meng Q.; Gong D.; Chung PWH. How Good Are Distributed Allocation Algorithms for Solving Urban Search and Rescue Problems? A Comparative Study With Centralized Algorithms. *IEEE Trans. Autom. Sci. Eng.* 2019;16(1), 478–485. doi: [10.1109/TASE.2018.2866395](https://doi.org/10.1109/TASE.2018.2866395).
- Chen Y.; Ren T.; Li Y.; Jiang G.; Liu Q.; Chen Y.; Yang SX. AI-empowered intelligence in industrial robotics: technologies, challenges, and emerging trends. *Intell. Robot.* 2026;6(1), 1–18. doi: [10.20517/ir.2026.01](https://doi.org/10.20517/ir.2026.01).
- Li J.; Yang SX. Embodied artificial intelligence as a paradigm shift for human–robot collaboration. *Intell. Robot.* 2026;6(1), 97–100. doi: [10.20517/ir.2026.05](https://doi.org/10.20517/ir.2026.05).
- Alsharafa NS.; Elangovan K.; Arulmozhiselvan L.; Ganiya RK.; Smerat A.; Ayasrah FT.; Florence MMV.; Sengan S. Optimising human-robot collaborative teleoperation using adaptive fuzzy logic control and real-time motion intention estimation. *Intell. Robot.* 2026;6(1), 120–147. doi: [10.20517/ir.2026.07](https://doi.org/10.20517/ir.2026.07).
- Huang B.; Yang Y.; Tsai YY.; Yang GZ. A Reconfigurable Multirobot Cooperation Workcell for Personalized Manufacturing. *IEEE Trans. Autom. Sci. Eng.* 2022;19(3), 2581–2590. doi: [10.1109/TASE.2021.3092560](https://doi.org/10.1109/TASE.2021.3092560).
- Chen J.; Sun D. Coalition-Based Approach to Task Allocation of Multiple Robots With Resource Constraints. *IEEE Trans. Autom. Sci. Eng.* 2012;9(3), 516–528. doi: [10.1109/TASE.2012.2201470](https://doi.org/10.1109/TASE.2012.2201470).
- Zhang A.; Yang M.; Bi W.; Gao F. Distributed Task Allocation With Critical Tasks and Limited Capacity. *Robotica* 2021;39(11), 2008–2032. doi: [10.1017/S0263574721000102](https://doi.org/10.1017/S0263574721000102).
- Huang P.; Zeng L.; Chen X.; Luo K.; Zhou Z.; Yu S. Edge Robotics: Edge-Computing-Accelerated Multirobot Simultaneous Localization and Mapping. *IEEE Internet Things J.* 2022;9(15), 14087–14102. doi: [10.1109/JIOT.2022.3146461](https://doi.org/10.1109/JIOT.2022.3146461).
- Li L.; Chen Z.; Kan Z. Environment-Driven and LLM-Guided Multi-Robot Task Inference and Allocation Under Temporal Logic Specifications. *IEEE Trans. Autom. Sci. Eng.* 2026;23, 2925–2940. doi: [10.1109/TASE.2026.3659055](https://doi.org/10.1109/TASE.2026.3659055).
- Zhang H.; Pan Q.; Miao Z.; Zhu B. An Effective Iterated Greedy Algorithm for Multi-Robot Task Allocation and Scheduling Problem. In: 2025 5th International Conference on Computer, Control and Robotics (ICCCR). Hangzhou, China: IEEE; 2025. pp. 105–109. doi: [10.1109/ICCCR65461.2025.11072662](https://doi.org/10.1109/ICCCR65461.2025.11072662).
- Wang S.; Liu Y.; Qiu Y.; Li S.; Zhou J. An Efficient Distributed Task Allocation Method for Maximizing Task Allocations of Multirobot Systems. *IEEE Trans. Autom. Sci. Eng.* 2024;21(3), 3588–3602. doi: [10.1109/TASE.2023.3281577](https://doi.org/10.1109/TASE.2023.3281577).
- Nayak J.; Mondal S.; Saha S.; Sarkar C. Multi-Robot Task Allocation: Impact of Limited Information Sharing. In: 2025 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS). IEEE; 2025. doi: [10.1109/ANTS66931.2025.11429973](https://doi.org/10.1109/ANTS66931.2025.11429973).
- Yu Y.; Tang Q.; Jiang Q.; Fan Q. A Deep Reinforcement Learning-Assisted Multimodal Multiobjective Bilevel Optimization Method for Multirobot Task Allocation. *IEEE Trans. Evol. Comput.* 2025;29(3), 574–588. doi: [10.1109/TEVC.2025.3535954](https://doi.org/10.1109/TEVC.2025.3535954).
- Martin DR.; Butler BA.; Nivison S.; Egerstedt M.; Al Faruque MA.; Khargonekar P. Collaborative Task Allocation for Heterogeneous Multi-Robot Systems Through Iterative Clustering. *IEEE Robot. Autom. Lett.* 2026;11(1), 33–40. doi: [10.1109/LRA.2025.3632086](https://doi.org/10.1109/LRA.2025.3632086).
- Dai W.; Rai U.; Chiun J.; Cao Y.; Sartoretti G. Heterogeneous Multi-robot Task Allocation and Scheduling via Reinforcement Learning. *IEEE Robot. Autom. Lett.* 2025;10(3), 2654–2661. doi: [10.1109/LRA.2025.3534682](https://doi.org/10.1109/LRA.2025.3534682).
- Zhang L.; Zhao J.; Lamon E.; Wang Y.; Hong X. Energy Efficient Multi-Robot Task Allocation Constrained by Time Window and Precedence. *IEEE Trans. Autom. Sci. Eng.* 2025;22, 18162–18173. doi: [10.1109/TASE.2023.3312214](https://doi.org/10.1109/TASE.2023.3312214).